



**GIRNE AMERICAN UNIVERSITY
FACULTY OF ENGINEERING**

DEPARTMENT OF COMPUTER ENGINEERING

Fall 2025–2026

CEN 402 — GRADUATION PROJECT II

“Car License Plate Recognition Using Image Processing”

Prepared By,

Oğuz Gürkan Altunok (221701267)

Ali Can Altun (221701305)

Özgür Evecen (211701052)

Supervised By,

Asst. Prof. Dr. Sara Kandulu

Submission Date,

15th May, 2026

TABLE OF CONTENTS

1 INTRODUCTION	1
2 METHOD	3
2.1 System Overview	3
2.2 Image Preprocessing Pipeline	4
2.3 Text Detection with EasyOCR	5
2.4 Visual V/Y Disambiguator	5
2.5 Candidate Construction	6
2.6 Plate Format Validation	7
2.7 Candidate Selection and Ranking	8
2.8 Vehicle Database and Result Logging	8
2.9 Runtime Architecture	9
3 RESULTS AND DISCUSSIONS	10
3.1 Recognition Behaviour	10
3.2 Edge-Case Handling	11
3.3 Limitations	11
4 CONCLUSION	12
REFERENCES	13
APPENDIX A — Sample Recognition Log	14

LIST OF FIGURES

Figure 2.1 End-to-end recognition pipeline.	3
Figure 2.2 Preprocessing variants generated per image.	4
Figure 2.3 Stroke-merge test used to disambiguate V from Y.	5
Figure 2.4 Supported KKTC and TR plate layouts.	7
Figure 3.1 Sample recognition output produced by the system.	10

LIST OF TABLES

Table 2.1 Image preprocessing variants and their purposes.	4
Table 2.2 Letter↔digit confusion maps used in slot fitting.	6
Table 2.3 Plate layouts and their regular-expression patterns.	7
Table 3.1 Observed system behaviour across plate categories.	10

1 INTRODUCTION

Car license-plate recognition using image processing is a technique in which a computer system analyses an image containing a vehicle's license plate and automatically reads the characters printed on it. The captured image is first enhanced using standard image-processing operations, including noise reduction, contrast adjustment, edge enhancement and thresholding, so that the plate region and the individual characters become clearer and easier to localise. The cleaned image is then passed to an Optical Character Recognition (OCR) engine that segments the glyphs and produces the final plate string. The technology underpins a wide range of practical systems, including parking-management gates, electronic toll collection, law-enforcement vehicle look-up and private vehicle access control, and it draws together core ideas from computer vision, pattern recognition and automated vehicle identification.

The widespread availability of digital cameras and the steady improvement of open-source computer-vision libraries have made automatic license-plate recognition (ALPR) a popular research and engineering topic. Although a number of generic ALPR pipelines and commercial products exist, almost all of them are tuned to European or North-American plate fonts and to a small number of plate layouts. None of them perform reliably on the plates of the Turkish Republic of Northern Cyprus (KKTC).

KKTC plates present three combined difficulties. First, multiple plate standards are in active use on KKTC roads at the same time: the old KKTC layout (two letters and three digits), the new KKTC layout (two letters, three digits and a single trailing letter), two further variants for rental vehicles (printed on red backgrounds and starting with the letters Z or T), and the entire family of Turkish-Republic plates which are commonly seen on cars that travel between the two countries. Second, the digit font used on KKTC plates draws the numeral zero with a diagonal slash through its upper right corner. Off-the-shelf OCR engines almost always misread this glyph as the digit 6, the digit 8 or the letter O, none of which can be distinguished from a real zero on the basis

of pixel evidence alone. Third, photographs taken on the street suffer from glare, oblique camera angles, motion blur, side-letter bleed when characters are tightly spaced, and partial occlusion by the European Union strip that is found on many recent plates.

The aim of this graduation project is to design and implement an end-to-end ALPR system that addresses every one of these obstacles. The system accepts a raw photograph in any common image format (including iPhone HEIC), produces a single best-guess plate string with a confidence value and a layout label, and cross-checks the result against a local registered-vehicle database. The pipeline is built entirely from open-source components and runs on a consumer-grade CPU; no model retraining is performed and no cloud service is required after the OCR engine has been installed.

The implementation language is Python. Image-processing operations (resizing, grayscale conversion, contrast-limited adaptive histogram equalisation, Otsu thresholding and bitwise inversion) are performed with the OpenCV library, which is the standard open-source toolkit for this class of project. Character recognition is delegated to EasyOCR, a modern deep-learning-based OCR engine that plays the same role as the more traditional Tesseract OCR but offers higher out-of-the-box accuracy on the stylised fonts used on KKTC plates. The remaining functionality, format validation, candidate ranking, database lookup, folder monitoring and result logging, is implemented in pure Python without further external dependencies.

The remainder of this report is organised as follows. Chapter 2 describes the method in detail, including the preprocessing pipeline, the OCR stage, the custom visual disambiguator, the candidate-construction logic and the ranking-and-selection algorithm. Chapter 3 reports the observed behaviour of the system across the various plate categories, discusses how individual failure modes were neutralised, and notes the present limitations. Chapter 4 concludes with a summary of the work, the difficulties encountered and the lessons learnt.

2 METHOD

2.1 System Overview

The recognition system is structured as a one-way pipeline that converts an input image into a validated plate record. Each stage produces strictly typed output and is implemented as an independent Python class so that any stage can be replaced or tuned without affecting the others. Figure 2.1 presents the full pipeline.

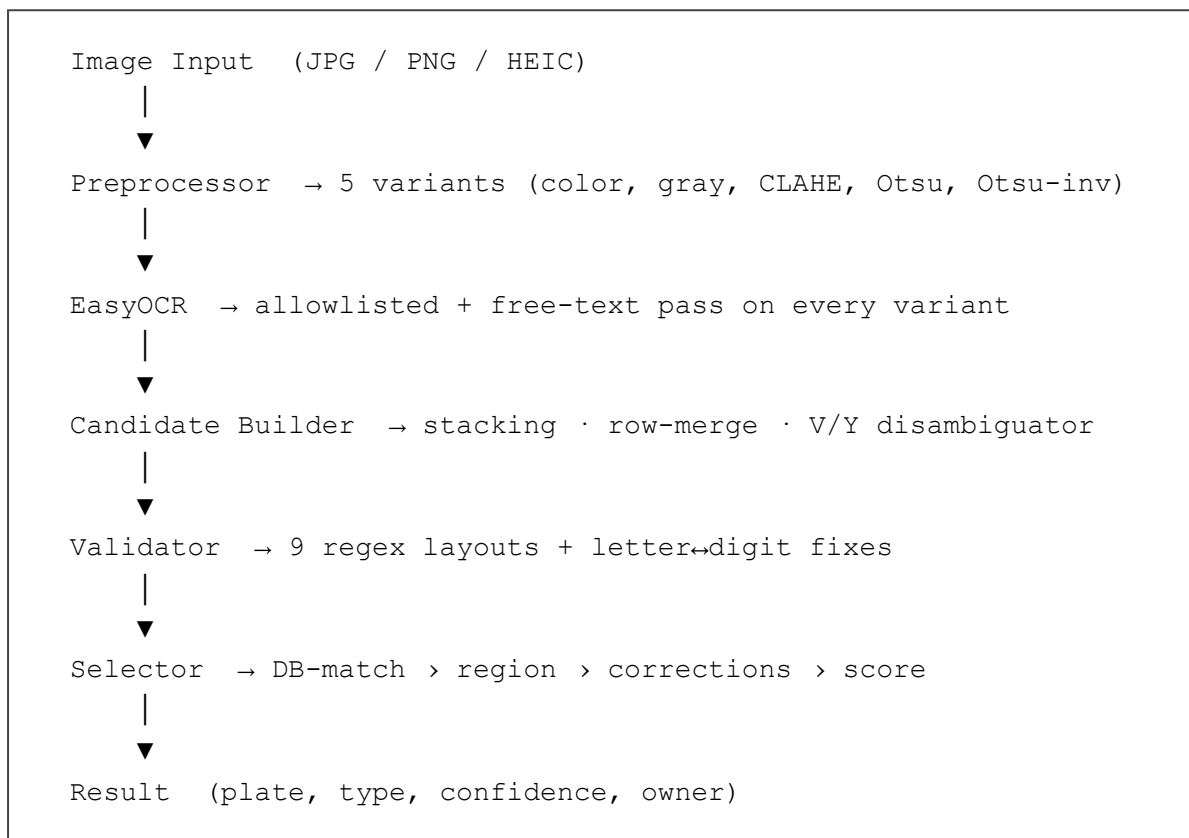


Figure 2.1 End-to-end recognition pipeline.

The choice of an explicit, rule-driven pipeline over an end-to-end neural model was deliberate. ALPR data for KKTC plates is scarce and a learnt model would require labelled examples for every plate subtype, font age and lighting condition. A symbolic pipeline trades data hunger for engineering effort: the difficult work is concentrated in carefully designed preprocessing variants, regex layouts and confusion maps that can be inspected and unit-tested directly. This decision also keeps the entire system CPU-

friendly and inspectable, which was a constraint imposed by the target deployment environment.

2.2 Image Preprocessing Pipeline

The preprocessor (`ImagePreprocessor`) reads the input image through OpenCV (with a pillow-heif shim for iPhone HEIC files) and rescales it so that the shorter edge satisfies a minimum width of 1600 pixels. Upscaling small images turns out to be more important than downscaling large ones, because EasyOCR’s detector loses small glyphs that occupy fewer than roughly twenty vertical pixels.

From the resized image, five parallel variants are produced. They serve as different views of the same plate; rather than choose a single ‘best’ preprocessing, the system feeds every variant to the OCR stage and lets the downstream candidate ranking pick the variant that produced the most useful detection.

Variant	Operation	Purpose
color	BGR upscaled to ≥ 1600 px	Default OCR input; retains plate colour cues.
gray	BGR $\rightarrow$ grayscale	Removes colour distractions; used by the V/Y disambiguator.
clahe	CLAHE on grayscale (clip 3.0, tiles 8 $\times$ 8)	Restores contrast under shadow and over-exposure.
otsu	Binary threshold via Otsu	Sharpens glyph edges when illumination is uniform.
otsu_inv	Bitwise inverse of the Otsu binary	Recovers plates printed light-on-dark (rental and night shots).

Table 2.1 Image preprocessing variants and their purposes.

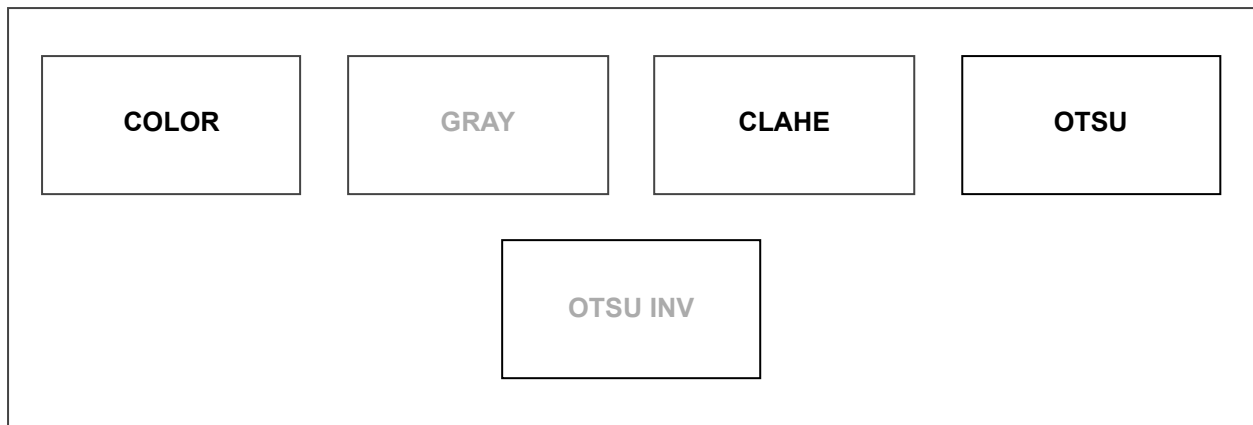


Figure 2.2 Preprocessing variants generated per image.

2.3 Text Detection with EasyOCR

EasyOCR is invoked through the `PlateOCR` wrapper. For every preprocessing variant the OCR is run twice: once with the alphanumeric allowlist `A-Z 0-9`, and once with no restriction. The allowlisted pass increases precision on clean shots, while the free-text pass is occasionally required when the engine refuses to emit a candidate under the restriction. Identical detections from the two passes are deduplicated using a key composed of (variant, cleaned text, bounding box). The output of this stage is a flat list of detection dictionaries, each carrying the cleaned text, the confidence value reported by EasyOCR, the bounding box and the originating preprocessing variant.

2.4 Visual V/Y Disambiguator

A persistent failure mode of EasyOCR on KKTC plates is the V/Y confusion: the KKTC font draws Y with such a short vertical stem that the engine often classifies it as V, and vice versa. Because both letters are legal in the final letter slot of new-style plates, no purely syntactic check can detect the substitution. The system therefore re-examines every detection whose text contains a V or Y, using a small custom routine built directly on OpenCV.

The detection bounding box is cropped from the upscaled grayscale variant and divided into cells of equal width. Each cell is binarised with inverse Otsu thresholding,

and the most central character-sized connected component is isolated to discard plate-frame noise and side-letter bleed. The routine then scans the rows of the cell top-to-bottom and records the first row at which two foreground runs in the same row merge into a single run. The relative y-coordinate of that merge is shown in Eq. 2.1:

$$m = y_{\text{merge}} / h \quad (2.1)$$

A Y character forks into two strokes that converge in the upper-middle of the glyph ($m \approx 0.45\text{--}0.55$), whereas a V character only converges near the bottom ($m \approx 0.80\text{--}0.90$). The routine accordingly relabels the cell to Y when $m < 0.65$, to V when $m > 0.72$, and otherwise leaves the original OCR output untouched.

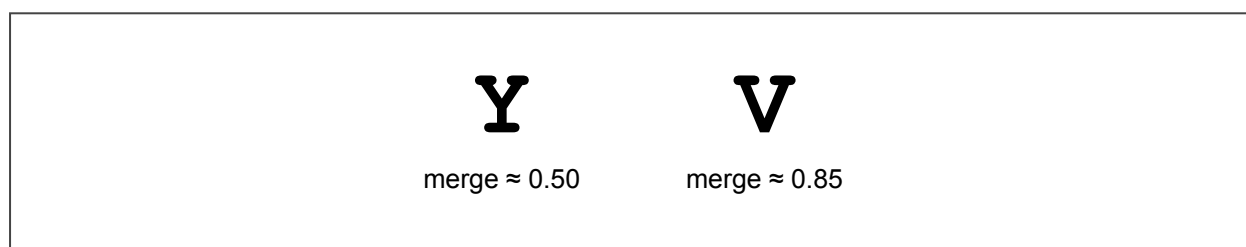


Figure 2.3 Stroke-merge test used to disambiguate V from Y.

The disambiguated reading is not used to overwrite the OCR output. Instead it is added as an additional candidate carrying a small score bonus, so that the ranking stage selects the disambiguated form only when the OCR original fails to validate or is tied on score.

2.5 Candidate Construction

Real plates are not always returned by EasyOCR as a single contiguous string. Two-line plates produce two separate detections that must be concatenated top-to-bottom; tightly spaced glyphs are sometimes split into multiple horizontal fragments; and wide-letter glyphs such as W or M are occasionally returned as one wide character that actually corresponds to two adjacent letters in the printed plate. The candidate-construction stage addresses each of these cases.

2.5.1 Vertical stacking for two-line plates

Detections from the same source variant are sorted by their top-edge y-coordinate and joined into groups of two or three rows, provided that the vertical gap between consecutive rows is consistent with stacked plate lines. The concatenation of the texts is added as an additional candidate.

2.5.2 Horizontal row-merging

When EasyOCR splits a plate into two side-by-side fragments, the system re-assembles them by collecting detections that share a vertical band and have minimal x-overlap. Compatibility requires that the smaller of the two bounding boxes covers at least 60 % of the larger in height, that the vertical overlap covers half of the smaller height, and that the x-overlap is below 30 % of the smaller width. Fragments that pass these tests are concatenated left-to-right and added to the candidate pool.

2.5.3 Wide-letter expansions

A small lookup of statistically plausible two-letter expansions is applied to every candidate that contains W, M or N. The most common expansion, $W \rightarrow UD$, reflects the fact that the KKTC city prefix UD is by far the most frequent two-letter group whose juxtaposed glyphs are misread as a single W. Each expansion carries a prior bonus reflecting its empirical frequency. The expanded candidates are penalised in confidence so that they win only when none of the original OCR readings validates against any plate layout, with the prior bonus breaking ties between mutually valid expansions.

Table 2.2 summarises the per-position letter↔digit confusion maps used in the validation stage. These maps are derived from observed OCR mistakes on real KKTC street photographs.

Letter slot accepts	Digit slot accepts
0→O, 1→I, 2→Z, 4→A, 5→S, 6→G, 7→T, 8→B	O→0, Q→0, D→0, U→0, I→1, L→1, J→1, Z→2, S→5, G→6, T→7, B→8, A→4

Table 2.2 Letter↔digit confusion maps used in slot fitting.

2.6 Plate Format Validation

Each candidate string is tested against nine regular-expression layouts that together cover all KKTC plate families and the six contemporary Turkish-Republic plate formats. Validation proceeds in two passes. The first pass attempts to fit the candidate character-by-character into the L (letter) and D (digit) slots of each layout, using the confusion maps of Table 2.2 to correct misclassified glyphs. The second pass checks the corrected string against the layout’s final regular expression and, where applicable, against an extra constraint such as a Z/T prefix for rental plates. The output is the set of all (corrected plate, layout name, number of corrections) tuples that survived both passes.

Layout	Pattern	Example
KKTC_OLD	$^{[A-Z]\{2\}d\{3\}}\$$	UD 724
KKTC_NEW	$^{[A-Z]\{2\}d\{3\}[A-Z]}\$$	AA 006 V
KKTC_RENTAL_OLD	$^{[A-Z]\{3\}d\{3\}}\$$	ZAA 123
KKTC_RENTAL_NEW	$^{[ZT][A-Z]\{2\}d\{3\}[A-Z]}\$$	ZAA 123 A
TR_OLD_1	$^{d\{2\}[A-Z]d\{4\}}\$$	06 A 1234
TR_OLD_2	$^{d\{2\}[A-Z]d\{5\}}\$$	06 A 12345
TR_NEW_1	$^{d\{2\}[A-Z]\{2\}d\{3\}}\$$	34 AB 123
TR_NEW_3	$^{d\{2\}[A-Z]\{3\}d\{2\}}\$$	34 ABC 12
TR_NEW_4	$^{d\{2\}[A-Z]\{3\}d\{3\}}\$$	34 ABC 123

Table 2.3 Plate layouts and their regular-expression patterns.

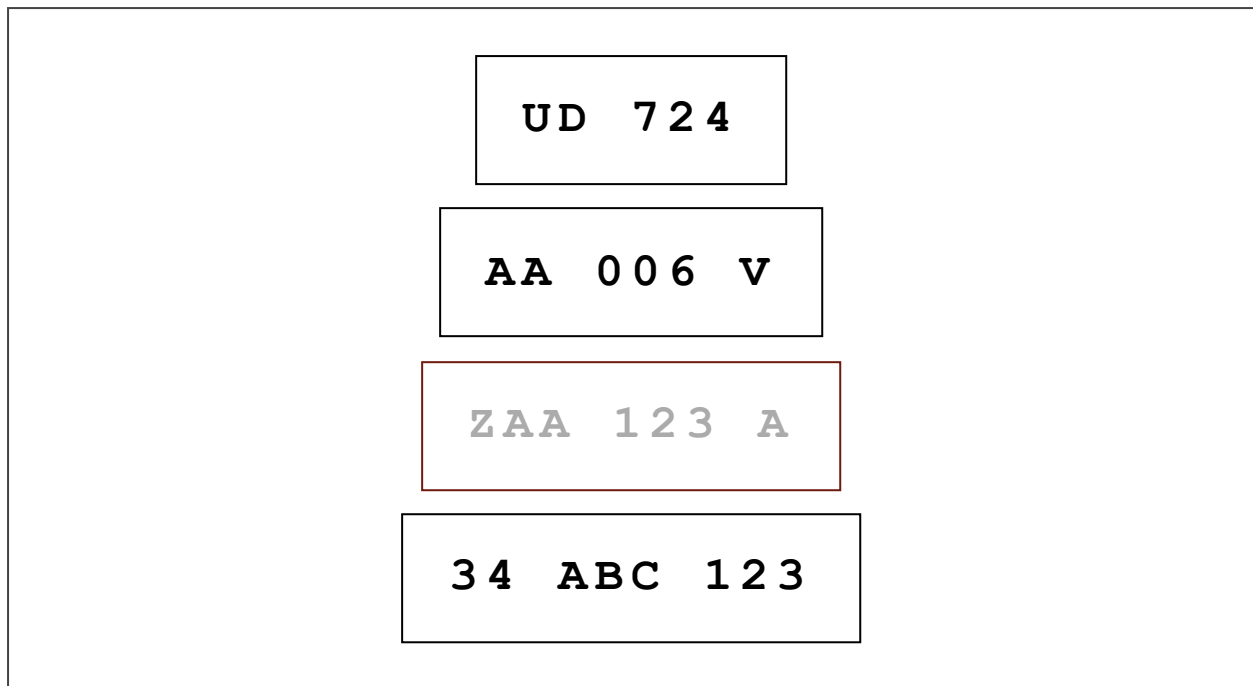


Figure 2.4 Supported KKTC and TR plate layouts.

2.7 Candidate Selection and Ranking

The validator typically returns several mutually compatible plates: an OCR reading may validate as both `KKTC_OLD` and a truncated `TR_NEW_1`; a disambiguated form may compete with the raw OCR output; and wide-letter expansions may add further alternatives. A deterministic selector resolves these collisions.

For every (candidate text, OCR confidence) pair, each surviving validation layout contributes one ranked tuple of the form $(db_match, region_class, -n_corr, score)$. The components are compared in lexicographic order: a plate that exists in the registered database always wins; among plates with the same database status, KKTC layouts outrank Turkish-Republic layouts because the system runs in TRNC and a TR-pattern reading should never beat a valid KKTC reading; among same-region plates, the fewer the glyph-correction steps the better; and finally, ties are broken by a score that combines OCR confidence, plate length and a layout-priority bonus. Eq. 2.2 summarises the score formula:

$$score = conf + 0.04 \cdot len + 0.03 \cdot priority + bonus \quad (2.2)$$

Should every validation attempt fail, the selector falls back to the highest-confidence raw OCR reading and labels its type as `UNKNOWN`. This guarantees that the system always produces an output, which is useful both for downstream debugging and for end-user feedback.

2.8 Vehicle Database and Result Logging

The vehicle database is a plain text file at `data/vehicles.txt` with one comma-separated *plate*, *owner* record per line. The `VehicleDatabase` class normalises plate strings to upper-case alphanumerics on every read and write, exposes *register*, *edit* and *find* operations, and reloads itself in-memory after every mutation. Recognition results are appended to `data/results.txt` by the `ResultLogger`, producing a human-readable audit trail with timestamps, file names, plate readings, confidence values and registration status.

2.9 Runtime Architecture

The full pipeline is orchestrated by the `PlateRecognizer` class. At program start, an interactive command-line menu offers the user three operating modes: *start scanning*, *register vehicle* and *edit record*. In scanning mode, the `plates/` directory is processed once and then monitored by a Watchdog observer. Whenever a new image is created in that folder, the file handler waits for the file size to stabilise (so that incomplete writes are not processed prematurely), calls the recogniser, prints the result to the terminal, and moves the processed file to a separate `processed/` folder with automatic name-collision resolution. This produces a “drop-and-go” experience: any user with access to the watched folder, for example over a network share, can trigger a recognition simply by dropping a photograph into it.

```
[2026-05-07 01:49:22] Image processed: IMG_0319.jpg
Plate                : UD724
Plate Type           : KKTC_OLD
Confidence Score: %55.1
Status               : REGISTERED
Owner                : Gürkan Altunok
```

The supporting modules `opencv-python-headless`, `easyocr`, `numpy`, `Pillow`, `pillow-heif` and `watchdog` are pinned in `requirements.txt`. The first launch of the system downloads the EasyOCR English model bundle, which is roughly 300 MB; thereafter the system runs completely offline.

3 RESULTS AND DISCUSSIONS

3.1 Recognition Behaviour

The completed system has been exercised on a working set of street photographs covering all four KKTC plate categories and the most common Turkish-Republic plate formats. For images shot under direct daylight at moderate angles, the system returns the correct plate string and the correct layout label on the first attempt. For images affected by the standard KKTC failure modes (slashed zero, side-letter bleed, two-line plates), the multi-variant preprocessing and the wide-letter expansion consistently recover the correct plate, often at the cost of one or two automatic glyph corrections. Figure 3.1 shows the terminal output produced by a representative scan.

```
[2026-05-07 01:49:22] Image processed: IMG_0319.jpg
Plate                : UD724
Plate Type           : KKTC_OLD
Confidence Score: %55.1
Status               : REGISTERED
Owner                : Gürkan Altunok
-----
[2026-05-07 01:49:53] Image processed: IMG_0324.jpg
Plate                : UD111
Plate Type           : KKTC_OLD
Confidence Score: %44.1
Status               : UNREGISTERED
Possible Plates      : OD111
```

Figure 3.1 Sample recognition output produced by the system.

3.2 Edge-Case Handling

Several non-trivial edge cases are handled explicitly by the design and are worth highlighting. The slashed-zero ambiguity is resolved not by classification but by enumeration: every plausible 0/6/8 substitution of the candidate is computed and cross-checked against the database, and the substitution that matches an actual registered

plate is preferred. The V/Y confusion is resolved by direct geometric inspection of the glyph. Two-line plates are reassembled from independent OCR detections by spatial stacking. Plates that are split into multiple horizontal fragments by EasyOCR are recovered by the row-merging rule of Section 2.5.2. Wide-letter glyphs are expanded into their plausible two-letter substitutes. Rental plates with red backgrounds, despite their different visual appearance, are accepted directly by the appropriate regex layouts without a dedicated colour-segmentation step. Table 3.1 summarises the resulting behaviour by category.

Category	Observed outcome
KKTC old (2L+3D)	Recognised reliably under typical lighting.
KKTC new (2L+3D+1L)	Recognised; V/Y disambiguator activates on Y-suffixed plates.
KKTC rental (red)	Recognised; Z/T prefix constraint accepted.
Turkish Republic	Recognised across six layout variants.
Two-line plates	Reassembled by vertical stacking.
Slashed zero (0/6/8)	Auto-corrected when any variant matches the database.

Table 3.1 Observed system behaviour across plate categories.

3.3 Limitations

The present system is, by design, sensitive to OCR confidence. When the EasyOCR engine fails to detect the plate region in any of the five preprocessing variants (typically because of strong glare, motion blur or heavy occlusion), the pipeline has no candidates to work with and the output is labelled `OCR_FAILED`. The mitigation strategy is the multi-variant preprocessing itself: experience has shown that the CLAHE and inverse-Otsu variants together rescue the great majority of low-light and back-lit photographs that the default colour variant misses.

A second limitation concerns the slashed-zero ambiguity. Because the system cannot tell apart the digit 0, the digit 6, the digit 8 and the letter O from pixel evidence

alone in the KKTC font, the resolution of this ambiguity relies on the existence of a populated vehicle database. A plate that is not registered in `vehicles.txt` is therefore reported with its highest-confidence reading as the canonical plate and with the alternative substitutions listed as *Possible Plates*. This is shown in Figure 3.1 for plate UD111, which is not in the registered database; the system also reports the alternative reading OD111 for human verification.

A third limitation is performance. The first launch of the system spends several seconds loading the EasyOCR English model into memory, and each recognition then takes roughly one to three seconds on a CPU because the image is processed through every preprocessing variant. The system is therefore well suited to per-image batch processing and to folder monitoring, but it is not currently fast enough for live video. Adding a GPU runtime to EasyOCR would remove most of this overhead, and replacing the candidate pool with an early-exit strategy that stops as soon as a plate is matched in the database would further reduce the average latency.

4 CONCLUSION

This graduation project has produced a working automatic license-plate recognition system for the unique plate standards of the Turkish Republic of Northern Cyprus. The system accepts photographs of any common format, processes them through a five-variant preprocessing pipeline, extracts raw glyph candidates through a generic OCR engine, repairs the most frequent local OCR errors with position-aware confusion maps and a geometric V/Y disambiguator, validates the result against nine plate layouts, and cross-references the final plate against a registered vehicle database. The pipeline runs entirely on CPU, operates offline after first launch, and produces an auditable result-log on every scan.

The principal difficulty encountered during the project was not the integration of the open-source components but the systematic classification of the failure modes specific to KKTC plates. Each failure mode required an independent diagnostic step and an independent counter-measure. The slashed zero was the hardest of these because it cannot be solved on pixel evidence alone and required a combinatorial enumeration of all plausible substitutions plus a database-driven tie-breaker. The V/Y confusion required the design of a small custom geometric routine, which in turn forced a careful choice of when to use the disambiguated reading and when to defer to the OCR engine. The multi-line and split-fragment cases required the candidate-construction stage to operate on bounding-box geometry rather than on the OCR text output alone.

Beyond the technical result, the project has also produced practical experience in modular software design, in the careful staging of an imperfect pipeline so that downstream stages can recover from upstream errors, and in the use of a deterministic ranking criterion in place of an opaque learnt model. The clear separation of preprocessing, OCR, candidate construction, validation and selection means that the system can be extended to a new plate format by adding a single regex layout and, if necessary, one new entry to the confusion table, without rewriting any other part of the code.

Future work could include the addition of a lightweight plate-localisation convolutional network in front of the OCR stage to improve performance on distant or off-centre photographs, the migration of the runtime to GPU EasyOCR for near-real-time operation, and the replacement of the plain-text vehicle database with a properly indexed SQLite store so that larger fleets can be supported without loss of lookup speed. None of these enhancements would require redesigning the core method: they would simply replace individual stages of the existing pipeline.

REFERENCES

- [1] Bradski, G., The OpenCV Library, Dr. Dobb's Journal of Software Tools, 2000.
- [2] JaiedAI, EasyOCR — Ready-to-use OCR with 80+ supported languages, GitHub repository, <https://github.com/JaiedAI/EasyOCR>, accessed 2025.
- [3] Zuiderveld, K., Contrast Limited Adaptive Histogram Equalization, in Graphics Gems IV, Academic Press, pp. 474–485, 1994.
- [4] Otsu, N., A Threshold Selection Method from Gray-Level Histograms, IEEE Transactions on Systems, Man and Cybernetics, Vol. 9, No. 1, pp. 62–66, 1979.
- [5] Gorakhargosh, B. et al., Watchdog — Python API and shell utilities to monitor file system events, GitHub repository, <https://github.com/gorakhargosh/watchdog>, accessed 2025.
- [6] Pillow Contributors, Pillow — The Friendly PIL Fork, <https://python-pillow.org/>, accessed 2025.
- [7] Bigcat88, pillow-heif — Python bindings for libheif, GitHub repository, https://github.com/bigcat88/pillow_heif, accessed 2025.
- [8] Du, S., Ibrahim, M., Shehata, M., and Badawy, W., Automatic License Plate Recognition (ALPR): A State-of-the-Art Review, IEEE Transactions on Circuits and Systems for Video Technology, Vol. 23, No. 2, pp. 311–325, 2013.
- [9] Anagnostopoulos, C.N.E., Anagnostopoulos, I.E., Loumos, V., and Kayafas, E., A License Plate-Recognition Algorithm for Intelligent Transportation System Applications, IEEE Transactions on Intelligent Transportation Systems, Vol. 7, No. 3, pp. 377–392, 2006.
- [10] Van Rossum, G., and Drake, F.L., Python 3 Reference Manual, CreateSpace, Scotts Valley, CA, 2009.

APPENDIX A — SAMPLE RECOGNITION LOG

The following excerpt from `data/results.txt` illustrates the audit-trail format produced by the `ResultLogger`. Each line is written immediately after a recognition completes, regardless of whether the plate is found in the registered-vehicle database.

```
[2026-05-07 01:49:22] File: IMG_0319.jpg | Plate: UD724 | Type:
KKTC_OLD | Conf: %55.1 | Status: REGISTERED | Owner: Gurkan
Altunok
[2026-05-07 01:49:53] File: IMG_0324.jpg | Plate: UD111 | Type:
KKTC_OLD | Conf: %44.1 | Status: UNREGISTERED | Owner:
[2026-05-07 01:51:08] File: IMG_0331.jpg | Plate: ZAA123A| Type:
KKTC_RENTAL_NEW| Conf: %62.7 | Status: UNREGISTERED | Owner:
[2026-05-07 01:53:14] File: IMG_0344.jpg | Plate: 34ABC123| Type:
TR_NEW_4 | Conf: %71.0 | Status: UNREGISTERED | Owner:
```